

ПЗ №13. Наследование и полиморфизм в Python

Тема

Изучение механизмов наследования и полиморфизма в объектно-ориентированном программировании на языке Python.

Цель работы

- **Теоретическая:** Изучить принципы наследования, переопределения методов и полиморфизма; понять, как создавать иерархии классов.
- **Практическая:** Приобрести навыки создания базового класса с методами-заглушками и двух производных классов, переопределяющих эти методы; научиться использовать полиморфизм для работы с объектами разных классов через единый интерфейс.

Задачи

1. Создать базовый класс, содержащий методы, предназначенные для переопределения в потомках.
2. Реализовать два класса-наследника, которые по-своему реализуют унаследованные методы.
3. Продемонстрировать полиморфное поведение: обработать список объектов базового класса, вызывая их методы, и наблюдать разные результаты.

Оборудование и программное обеспечение

- **Программное обеспечение:** Python 3.8+, VS Code (или другая IDE).
- **Оборудование:** Персональный компьютер.

Краткие теоретические сведения

Наследование – механизм, позволяющий создать новый класс на основе существующего. Новый класс (производный, дочерний) получает все атрибуты и методы базового (родительского) класса и может расширять или изменять их.

```
class BaseClass:
    def method(self):
        print("Метод базового класса")
```

```
class DerivedClass(BaseClass):  
    def method(self):          # переопределение метода  
        print("Метод производного класса")
```

Полиморфизм – способность объектов с одинаковым интерфейсом иметь различную реализацию. В Python полиморфизм проявляется, когда мы вызываем один и тот же метод у объектов разных классов, и каждый объект выполняет свою версию метода.

Абстрактный метод – метод базового класса, который не выполняет полезных действий, а служит лишь шаблоном. В Python его можно создать с помощью `pass` или выбросить исключение `NotImplementedError`. В данной работе мы используем простой `pass` или заглушку, возвращающую ноль/пустую строку, чтобы показать намерение переопределить метод в потомке.

Порядок выполнения работы

1. Подготовительный этап

1. Запустите VS Code.
2. Создайте новый файл `lab_13_[ВашаФамилия].py`.
3. В начале файла укажите свои данные (имя, группа) в комментарии.

2. Основной этап

Задание 1. Общий пример (выполняется всеми студентами)

Изучите приведённый ниже код, реализующий простую иерархию классов «Животные». Скопируйте его в свой файл. Обратите внимание на то, как базовый класс `Animal` определяет методы-заглушки, а классы `Dog` и `Cat` переопределяют их. Затем создайте список из разных животных и вызовите их методы в цикле, чтобы увидеть полиморфизм в действии.

```
# Базовый класс  
class Animal:  
    def __init__(self, name):  
        self.name = name  
  
    def speak(self):  
        # Заглушка, будет переопределена  
        pass  
  
    def move(self):
```

```
# Заглушка
pass

def info(self):
    print(f"Я животное по имени {self.name}")

# Класс-наследник Dog
class Dog(Animal):
    def speak(self):
        print(f"{self.name} говорит: Гав-гав!")

    def move(self):
        print(f"{self.name} бегает на четырёх лапах")

    def info(self):
        print(f"Я собака по имени {self.name}")

# Класс-наследник Cat
class Cat(Animal):
    def speak(self):
        print(f"{self.name} говорит: Мяу-мяу!")

    def move(self):
        print(f"{self.name} крадётся бесшумно")

    def info(self):
        print(f"Я кошка по имени {self.name}")

# Демонстрация полиморфизма
animals = [Dog("Шарик"), Cat("Мурка"), Dog("Бобик")]

for animal in animals:
    animal.info()
    animal.speak()
    animal.move()
    print("-" * 20)
```

Что нужно сделать: Запустите этот код, убедитесь, что каждый объект ведёт себя по-своему. Попробуйте добавить ещё один класс-наследник (например, `Bird`) с собственной реализацией методов.

Задание 2. Самостоятельная разработка иерархии классов по вариантам

Выберите свой вариант и реализуйте описанную иерархию классов. Для каждого варианта:

- Создайте базовый класс с указанными методами (методы могут быть заглушками – использовать `pass` или возвращать 0/None).
- Создайте два класса-наследника, которые переопределяют все методы базового класса (каждый наследник должен иметь свою реализацию).
- При необходимости добавьте в классы атрибуты (например, название, размеры, скорость и т.п.) и инициализируйте их в конструкторе `__init__`.

Вариант 1. Геометрические фигуры

Базовый класс `Shape` :

- метод `area()` – возвращает 0 (заглушка)
 - метод `perimeter()` – возвращает 0
 - метод `info()` – выводит строку "Это геометрическая фигура"
- Классы-наследники: `Rectangle` (прямоугольник) и `Circle` (круг).
- `Rectangle` принимает ширину и высоту, переопределяет `area()` (возвращает площадь), `perimeter()` (периметр) и `info()` (выводит тип фигуры и её размеры).
 - `Circle` принимает радиус, переопределяет `area()` (πr^2), `perimeter()` (длина окружности) и `info()`.

Вариант 2. Транспортные средства

Базовый класс `Vehicle` :

- метод `move()` – выводит "Транспортное средство движется"
 - метод `fuel_consumption()` – возвращает 0 (расход топлива на 100 км)
 - метод `info()` – выводит "Это транспортное средство"
- Классы-наследники: `Car` (легковой автомобиль) и `Bicycle` (велосипед).
- `Car` имеет атрибуты: марка, расход топлива (например, 8 л/100км). `move()` выводит "Автомобиль едет по дороге", `fuel_consumption()` возвращает указанный расход, `info()` выводит марку и тип.
 - `Bicycle` имеет атрибут "тип велосипеда". `move()` выводит "Велосипед едет по велосодорожке", `fuel_consumption()` возвращает 0, `info()` выводит тип.

Вариант 3. Работники компании

Базовый класс `Employee` :

- метод `work()` – выводит "Сотрудник работает"

- метод `get_salary()` – возвращает 0
- метод `position()` – выводит "Должность не указана"
Классы-наследники: `Manager` и `Developer`.
- `Manager` имеет атрибуты: имя, оклад (например, 5000). `work()` – "Менеджер управляет проектом", `get_salary()` – возвращает оклад, `position()` – выводит "Менеджер".
- `Developer` имеет атрибуты: имя, оклад (например, 4000). `work()` – "Разработчик пишет код", `get_salary()` – возвращает оклад, `position()` – выводит "Разработчик".

Вариант 4. Электронные устройства

Базовый класс `Device`:

- метод `turn_on()` – выводит "Устройство включено"
- метод `turn_off()` – выводит "Устройство выключено"
- метод `info()` – выводит "Это электронное устройство"
Классы-наследники: `Laptop` и `Smartphone`.
- `Laptop` имеет атрибуты: модель, время работы от батареи. Переопределяет `info()` (выводит модель и тип), добавляет метод `work()` (выводит "Ноутбук работает от батареи").
- `Smartphone` имеет атрибуты: модель, размер экрана. Переопределяет `info()`, добавляет метод `call()` (выводит "Смартфон звонит").

Задание 3. Проверка работоспособности

Для своего варианта:

1. Создайте **минимум два объекта** каждого из классов-наследников (всего 4 объекта) с разными параметрами.
2. Поместите все объекты в один список (например, `objects`).
3. В цикле пройдите по списку и для каждого объекта вызовите **все три метода**, определённые в базовом классе (например, `area()`, `perimeter()`, `info()` – для фигур). Убедитесь, что каждый объект выполняет свою версию метода.
4. Вывод программы должен быть информативным: предваряйте вывод пояснениями.

Пример оформления проверки (для варианта 1):

```
shapes = [
    Rectangle(5, 3),
    Circle(2),
    Rectangle(10, 1),
```

```
        Circle(5)
    ]

    for shape in shapes:
        shape.info()
        print(f"    Площадь: {shape.area():.2f}")
        print(f"    Периметр: {shape.perimeter():.2f}")
        print("-" * 25)
```

3. Контрольный этап

1. Проверьте, что код выполняется без синтаксических ошибок.
2. Убедитесь, что все методы корректно переопределены и вывод соответствует ожиданиям.
3. Обратите внимание: если метод не переопределён в наследнике, будет вызвана версия из базового класса (заглушка) – это не должно происходить в вашем коде.

Контрольные вопросы

1. Что такое наследование? Для чего оно используется?
2. Как в Python указать, что класс наследуется от другого класса?
3. Что такое переопределение метода? Приведите пример.
4. Объясните понятие полиморфизма. Как он проявляется в Python?
5. Можно ли вызвать метод базового класса из метода дочернего класса? Если да, то как?
6. Что произойдёт, если в дочернем классе не переопределить метод, который вызывается в программе?
7. Поддерживает ли Python множественное наследование? Если да, то как оно работает?
8. Какие преимущества даёт использование полиморфизма при проектировании программ?
9. Можно ли создать объект базового класса, если он содержит только методы-заглушки? Будет ли это полезно?
10. Как проверить, принадлежит ли объект определённому классу или его наследнику?

Содержание отчета

1. Тема, цель и задачи работы.
2. Код программы (включая общий пример и реализацию своего варианта) с комментариями.

3. Результаты выполнения (вывод программы – скриншот или текст).
4. Ответы на контрольные вопросы.
5. **Выводы по работе:** Что нового узнали о наследовании и полиморфизме? Как эти механизмы помогают в организации кода? Удалось ли достичь цели работы? Какие трудности возникли?